

and send maximum feeds—the only limitation on updating channels is an interval of 15 seconds. Once the channel is created with a suitable description, applicable fields and meta information, you can identify the read or write API keys under *Channel settings* and the key for channel meta operations in the *Profile* section.

Let's now look at how a REST client for connecting with ThingSpeak can be built.

Qt HTTP connectivity

In order to talk to any network server using the HTTP protocol, Qt provides the *QNetworkAccessManager* class, which has asynchronous methods like *get*, *post*, *put* and *deleteResource* to perform various REST operations. These methods take the *QNetworkRequest* object as an argument, as well as an additional argument for request payload in the form of *QByteArray* for POST, PUT operations. *QNetworkRequest* mainly contains the URL in the form of a *QUrl* object, which encapsulates the protocol, host name, port, path, query string, etc. The *QUrlQuery* class can be used to form a *QueryString* effectively. These methods return the response in the form of a *QNetworkReply*, which needs to be explicitly destroyed using the *deleteLater* method in the slot connected to the *finished* signal of *QNetworkAccessManager*.

Let's consider a channel with three fields representing temperature, humidity and pressure. Let us assume that the various API keys and channel numbers are initialised as follows:

```
QString RDKey = "XXXXXXXXXXXXXXXXXXXX";
QString WRKey = "XXXXXXXXXXXXXXXXXXXX";
QString CHKey = "XXXXXXXXXXXXXXXXXXXX";
QString CHNum = "xxxxxx";
```

To send a feed, we need to perform the POST method with the steps that follow, using the URL *https://api.thingspeak.com/update.json* and the following JSON data as request payload:

```
{
  "api_key": "XXXXXXXX XXXXXXXX",
  "field1": "25",
  "field2": "72",
  "field3": "990"
}
```

Step 1: Let's prepare the data for POST in JSON format, assuming that *tval*, *hval* and *pval* represent the temperature, humidity and pressure values in integer form.

```
QVariantMap feed;
feed.insert("api_key", WRKey);
feed.insert("field1", QVariant(tval).toString());
feed.insert("field2", QVariant(hval).toString());
```

```
feed.insert("field3", QVariant(pval).toString());
QByteArray payload=QJsonDocument::fromVariant(feed).toJson();
```

You may verify the prepared JSON data in QString format as follows:

```
QDebug() << QVariant(payload).toString();
```

Or you can display it in a *QLineEdit* or *QTextEdit* widget for testing purposes.

Step 2: Prepare the URL as follows:

```
QUrl myurl;
myurl.setScheme("http"); //https also applicable
myurl.setHost("api.thingspeak.com");
myurl.setPath("/update.json");
```

We are skipping the user name and password as we are using API keys for authentication. We are also skipping *setPort* as the service is running on the default port 80. You can verify the prepared URL as follows:

```
QDebug() << myurl.toString();
```

Step 3: Prepare the network request, as follows:

```
QNetworkRequest request;
request.setUrl(myurl);
request.setHeader(QNetworkRequest::ContentTypeHeader,
                  "application/json");
```

Step 4: Perform the POST operation using the following code in the slot connected to the *Publish PushButton* click.

```
QNetworkAccessManager *restclient; //in class
restclient = new QNetworkAccessManager(this); //constructor
QNetworkReply *reply = restclient->post(request, payload);
QDebug() << reply->readAll();
```

restclient can be declared in the *MainWindow* or the

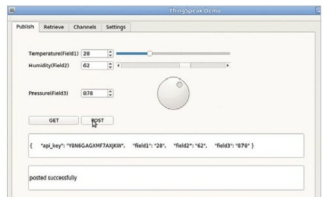


Figure 1: Sending a feed using POST